

Executive Summary

Предложенный инструмент миграции сейчас жёстко ориентирован на C# (Selenium/ NUnit) как источник и C#/TypeScript как цели. Он состоит из набора модулей (.NET CLI, ядра Migrator.Core, парсеров и генераторов) и использует **Roslyn** для разбора C#-кода. Основной подход – разбор тестов в AST C#, построение промежуточной модели (IR) действий (клики, ввод, ожидания и т.д.), применение пользовательских правил и генерация кода Playwright. При расширении на новые языки важно сохранить эти функции (сохранение смысловой модели теста, возможность ручной доработки через TODO, конфигурирование маппингов и профилей проекта). В анализе предложены пути абстрагирования (универсальное IR на основе AST или DSL), архитектурные паттерны (разделение фронтэнд/бэкэнд парсинга и генерации, паттерн «Посетитель» + «Генератор» [7†L775-L784]), а также поэтапный план расширения с учётом рисков и приоритетов. В рекомендациях указаны официальные парсеры/библиотеки для популярных языков (C#, Java, Python, TypeScript, Go, Rust) и сравнительные таблицы подходов (IR vs прямой трансляции vs шаблонов).

1. Текущая архитектура и C#-ориентированность

- **Модули:** проект включает **Migrator.Cli** (интерфейс командной строки и оркестратор), **Migrator.Core** (ядро, общий IR и конвейер миграции), **Migrator.SeleniumCSharp** (парсер тестов Selenium на C#), **Migrator.Roslyn** (низкоуровневый парсер C# с Roslyn), **Migrator.PlaywrightDotNet** и **Migrator.PlaywrightTypeScript** (генераторы кода для .NET и TypeScript).
- **API и компоненты:**
- *ITestFileParser* в ядре отвечает за разбор файлов в `TestFileModel` (IR теста). По умолчанию используется **RoslynTestFileParser**, который вызывает компилятор Roslyn для построения синтаксического дерева C#.
- *IActionExtractor* анализирует дерево C#-кода (набор распознавателей из Migrator.Roslyn) и выделяет действия Selenium (Click, SendKeys, Navigation, ожидания и т.д.).
- *IProjectAdapter* применяет правила переноса (страницы, методы) для преобразования исходных выражений в целевые. Текущий `DefaultProjectAdapter` жёстко рассчитан на схемы имен C# Page Object.
- *IRenderer* выполняет генерацию кода Playwright: существует `PlaywrightDotNetRenderer` и `PlaywrightTypeScriptRenderer` для C# и TS целей соответственно.
- **Связь с C#:** ключевая привязка – использование Roslyn (официальный компилятор C#) для разбора исходников и моделирования кода [23†L160-L168] . Вся логика анализа построена вокруг C#-конструкций (NUnit, Selenium WebDriver API и собственные модели действий). В архитектуре нет абстракции над парсингом других языков – отсутствует общий интерфейс для альтернативных фронтэндов (например, Java, Python).

2. Функциональные требования и сценарии (не потерять)

- **Полнота миграции:** инструмент должен продолжать корректно распознавать все ключевые конструкции Selenium-тестов (поиск элементов, клики, ввод, проверки, навигация, синхронизация) и перенаправлять их в эквивалент Playwright.

- **Сохранение моделей и TODO:** генерируемый код должен содержать пометки (TODO) для областей, требующих внимания (напр., сложные ожидания, неконвертируемые параметры) – чтобы не упустить необходимость ручного дооформления.
- **Поддержка профилей:** существующие конфигурации («profiles») отображений локаторов, параметров и имён методов нельзя потерять. Пользователь должен иметь возможность настраивать или добавлять собственные правила привязки имен (например, маппинг методов Page Object) [23†L160-L168] .
- **Совместимость проекта:** для генерации TypeScript-кода используется уже существующий проект Playwright TS (переданный через `--ts-project`). Нельзя терять возможность встраивать выходной код в текущую инфраструктуру (будильники CI, конфиги).
- **Интеграция тестов:** мигратор анализирует проект как целое, поэтому сценарии разбора директорий и скаффолдинга (scaffold) на основе структуры проекта надо сохранить.
- **Непрерывность миграции:** процессы «discover-target», «migrate», «verify» и другие режимы CLI должны остаться, чтобы не нарушать текущий рабочий сценарий (pipeline-конвейер и система отчётов).

3. Абстракции исходного/целевого кода (IR/AST/ универсальное представление)

- **Текущий IR:** мигратор уже использует внутрирефакторскую модель (TestFileModel в Migrator.Core.Models) – высокоуровневую IR теста, которая не привязана к конкретному языку-источнику и содержит действия (ClickAction, SendKeysAction, Assertions и т.д.). Это аналог AST-уровня для семантики теста. Например, JavaParser предоставляет «AST представление Java-кода» [25†L83-L92] , Python `ast` – синтаксическое дерево Python [27†L64-L72] , TypeScript Compiler API – AST TypeScript-файлов [31†L1-L2] .
- **Варианты абстракций:**
 - *Языко-специфичные AST:* использовать родной AST парсера (Roslyn для C#, JavaParser для Java, ast для Python, ts.SourceFile для TS [31†L1-L2] и т.д.), а затем переводить их в единый IR инструментом. Минус: придётся поддерживать множество фронтэндов.
 - *Универсальное IR:* проектировать обобщённый промежуточный формат (например, на основе JSON/DSL) со стандартными узлами (модель Page Object, действия, ожидания). Это позволит фронтэндам (разным языкам) конвертировать код в этот IR, а рендерерам – из него генерировать любой язык. Отсюда: как в компиляторах, общий IR (иерархический/графовый) связывает несколько фронтэндов и бэкэндов [3†L23-L32] [15†L52-L61] .
 - *Смешанный подход:* парсить исходник до AST, затем сразу генерировать целевой код с помощью шаблонов (skip явный IR). Такой подход (direct transpiler) проще для одного-двух языков, но не масштабируется на многоязычность и затрудняет внедрение новых оптимизаций. Для больших фреймворков предпочтительнее IR с отчётом о маппингах.

Цитата: «Путём перевода исходного кода в общее промежуточное представление компилятор гарантирует разделение обязанностей фронтэнда и бэкэнда» [15†L52-L61] . Это обеспечивает поддержку нескольких языков при том же IR [3†L23-L32] [15†L52-L61] .

4. Стратегии поддержки множества языков

- **Фронтэнд+бэкэнд:** сделать для каждого нового исходного языка собственный парсер/анализатор (фронтэнд), который производит внутренний IR. Аналогично, отдельный рендерер для каждого выходного языка. При таком подходе количество комбинаций растёт **линейно** ($O(n+m)$), а не экспоненциально, если изолировать. MongoDB Babel-плагин

описывает: «Для каждого входного языка один visitor, для каждого выходного – один генератор; для n языков нужна $O(n)$ работы, а не $O(n^2)$ » [7†L775-L784] .

- **Tree-sitter или гибридный парсер:** рассмотреть инструменты вроде Tree-sitter, которые поддерживают парсинг сразу многих языков. Тогда можно унифицировать «дерево» и ограничить число модулей парсинга. Но это достаточно рискованно: нужно обучить грамматики под нужные языки и маппинг, что может быть трудоёмко.
- **Шаблонное порождение:** для целевых языков можно использовать шаблонизаторы (например, T4 для C#, handlebars/EJS для TS), заполняя их данными IR. Такой подход хорошо для генерации повторяющегося кода (скелета тестов), но слаб в обработке сложной логики (подстановок, циклов). Часто его комбинируют с IR: сначала строится IR, потом простые паттерны переключаются в шаблоны.
- **Классический конвейер:** аналог компилятора: фронтэнд (парсинг AST исходника) → промежуточное представление (семантическое IR) → оптимизации (рефакторинг IR) → бэкэнд (генерация кода целевых языков). Такой подход модульный и расширяемый [15†L52-L61] .

5. Декомпозиция задач и приоритеты

1. **Анализ архитектуры (high):** полностью задокументировать текущие модули, API, связи (см. таблицу ниже). Определить точки «жёсткой привязки» к C# (Roslyn, специфичные API Selenium). Это основание для рефакторинга.
2. **Определение требований (high):** собрать полный список текущих функциональных сценариев (прикладные тесты, ограничения, отчётность) и фиксировать, что именно не должно пропасть (как упомянуто в п.2). Приоритезировать по наиболее критичным (тестовые проверки, основные действия Selenium→Playwright).
3. **Выбор абстракции (high):** разработать или выбрать общий IR/AST для мигратора. Возможные задачи: описать структуру IR (XML/JSON/классы объектов), обеспечить, чтобы он мог хранить семантику тестов из любого языка. Сравнить подходы (см. таблицу сравнения).
4. **Проектирование фронтэндов (medium):** определить нужные парсеры для новых языков-источников (Roslyn расширяется на Java или Python). Например, для Java использовать JavaParser [25†L83-L92] , для Python – модуль `ast` [27†L64-L72] , для Go – пакет `go/parser` [33†L195-L202] , для Rust – crate Syn [35†L99-L107] . Важно из каждого парсера получать одинаковый IR.
5. **Проектирование бэкэндов (medium):** аналогично, создать рендереры для новых целей: Java, Python, Go, Rust. Например, изучить возможности кодогенераторов или шаблонов (Roslyn же может генерировать C# AST и код, для TS – встроенный принтер). Установить общие интерфейсы IRenderer и адаптировать, следуя границам модулей.
6. **Интеграция и тестирование (high):** мигрировать существующие модульные тесты (TestFixtures) в новую структуру. Разработать интеграционные тесты, прогоняя полную миграцию на небольших образцах кода (все языки). Важно, чтобы «зеленый билд» был постоянным критерием завершения этапов (как описано в философии инструмента).
7. **CI/CD и качество (medium):** настроить непрерывную интеграцию, включающую запуск проверок кода, тестов миграции и линтеров. Обеспечить автоматическую проверку регрессий (например, сравнение артефактов до/после миграции, «config-diff» режим). Установить контроль покрытия (отчёты качества).

6. Предложения по рефакторингу

- **Паттерн «Фасад» для парсеров:** выделить общий интерфейс `ITestFileParser` и обеспечивать каждый язык-источник своим классом. Например, `JavaTestFileParser`

(JavaParser), `PythonTestFileParser` (`ast.parse`) и т.д. Так можно плавно добавлять новые без изменения остальной логики.

- **Посетитель + Генератор:** применять паттерн Visitor для обхода IR (или AST) и паттерн Generator для порождения кода. Как в MongoDB-примере: для каждого входного языка свой Visitor, для каждого выходного – свой Generator [7†L775-L784]. Благодаря этому ввод нового языка не затрагивает уже существующие.
- **Разделение ответственности:** утилиты общего назначения (например, обработка конфигов, чтение файлов) должны быть вынесены в отдельные службы (модули), не смешиваясь с логикой парсинга и генерации. Интерфейс CLI желательно минимизировать – лишь разбирает опции и вызывает ядро.
- **Покрывание тестами:** написать юнит-тесты на уровне IR и адаптеров (минимум C# → IR → C#/TS), а также end-to-end-тесты миграции. Использовать TestFixtures из существующего проекта. Внедрить regress/regression suite (например, тесты преобразования наборов простых тест-кейсов, сравнение с ожидаемым результатом).
- **CI/CD:** приёмка в GitLab/GitHub Actions – настроить несколько этапов: сборка, статический анализ кода (анализ качества C# и новых компонентов), юнит/интеграционные тесты. Добавить quality gates: прервать задачу, если есть незакрытые TODO (флаг `--fail-on-todo` уже в CLI). Использовать существующие шаблоны CI (см. `ci/gitlab-package.example.yml`).

7. Миграция тестов и регрессионные сценарии

- **Перенос существующих тестов:** текущие тесты (класс `MigrationPipelineTests`, `VerifyTests` и др. в проекте) нужно адаптировать под новую модульную структуру. Сохранив сценарии (парсинг, генерация, сравнение отчётов), но вызывая обновлённый конвейер.
- **Новые интеграционные тесты:** подготовить типовые проекты на всех целевых и исходных языках с известным набором тестов/страниц. Автоматически прогонять полную миграцию и сравнивать с ожидаемым шаблоном (для TS/C#/Java/... должна генерироваться корректная структура). Так выявляются изменения в API или утрате функций.
- **Тесты регрессии:** каждый релиз инструмента сопровождать автотестами, проверяющими, что изменения не сломали предыдущую функциональность (напр., прежние JSON-профили правильно применяются, новый код компилируется и проходит проверки). Рекомендуется включить проверки на целостность IR (нет потерянных действий).

8. Риски и зависимости

- **Парсинг и грамматики:** если расширять на новые языки, придётся опираться на сторонние парсеры (Roslyn, JavaParser, `ast`, `ts.Compiler`, `Go/parser`, `Syn`). Риск ошибок парсинга или несовместимостей версий (например, Go 1.26 AST изменён) требует детального тестирования.
- **Утрата информации:** при конвертации C#→IR→другой язык есть риск потерять семантику (порядок исполнения, специфичные API Selenium). Необходимо следить за тем, чтобы IR был полной моделью. Любые ограничения (например, функции LINQ или особенности синхронизации) должны фиксироваться как известные ограничения.
- **Сложность графа зависимостей:** при множестве входных/выходных языков быстро растёт комбинаторика. Решается модульной архитектурой (см. [7]). Главное – протестировать каждый «путь» (Java→Python, Python→Go и т.д.).
- **Отсутствие информации по инфраструктуре:** в задании не уточнены детали сборки, CI, объём кода. Предполагается, что текущий CI рассчитан только на .NET. Нужно уточнить, как и где будут билдиться новые языки (например, запуск Maven/Go tests) – это потенциальная зависимость.

- **Управление версиями:** каждое новое целевое окружение потребует управления версиями парсеров/SDK. Важно зафиксировать и документировать версии.

9. Примерный roadmap (фазы)



Временные оценки в каждой фазе – **high** для ключевых архитектурных и интеграционных работ, **medium** для рефакторинга кода и CI, **low** для написания документации и мелких улучшений.

10. Инструменты и библиотеки

- **C#:** парсинг – *Roslyn* (Microsoft .NET Compiler Platform) [23†L160-L168] ; генерация кода – тот же Roslyn API или T4-шаблоны.
- **Java:** разбор – *JavaParser* (open-source) [25†L83-L92] ; генерация – *javapoet* или *Eclipse JDT/ASTParser*. JavaParser поддерживает генерацию AST и кодогенерацию.
- **Python:** разбор – стандартный модуль `ast` [27†L64-L72] ; генерация – модуль `astor` или `black` для печати AST, либо шаблоны Jinja2.
- **TypeScript/JavaScript:** парсинг/AST – *TypeScript Compiler API* [31†L1-L2] (метод `ts.createSourceFile` и т.д.); можно использовать *Babel* (плагин TypeScript) или *Esprima/Acorn*; генерация – TS-принтер (`ts.Printer`) или *Prettier* для форматирования.
- **Go:** парсинг – пакет `go/parser` (стандартная библиотека) [33†L195-L202] ; AST-узлы – `go/ast` ; генерация – `go/format` + текстовые шаблоны.
- **Rust:** парсинг – библиотека *syn* [35†L99-L107] (экосистема Rust macros, даёт полный AST Rust); генерация – *quote* или `rustfmt` для форматирования, либо *procedural macros*.

Во всех случаях рекомендуется использовать официальные или широко поддерживаемые инструменты (Roslyn, JavaParser, TypeScript API, syn и т.д.), так как они актуализируются вместе с языками.

11. Таблицы и диаграммы

Сравнение подходов к трансляции кода

Подход	Преимущества	Ограничения
IR/AST (абстрактный)	Позволяет унифицировать обработку разных языков; облегчает анализ/оптимизацию; разделяет фронтэнд/бэкэнд [15†L52-L61] . Хорош для масштабируемости.	Требует дизайна IR и его поддержки; сложнее в реализации; возможна потеря некоторых языковых «сахаров».
Прямая трансляция	Проще реализовать на старте (замена шаблонов, регэкспы); быстро работает на ограниченном наборе.	Сильно не гибка: трудно поддерживать большое число языковых комбинаций; легко допустить ошибки; плохо масштабируется при росте функционала.
Шаблонная генерация	Удобно генерировать повторяющийся код (скелет тестов); можно быстро править выходные шаблоны; легко интегрируется с IR.	С трудом обрабатывает сложную логику (если блок шаблона велик или динамичен); шаблоны могут быть неочевидны для сложных случаев.

Модули/компоненты и их ответственности

Модуль	Ответственность	Зависимости
<i>Migrator.Cli</i>	Парсинг CLI-параметров, запуск команд (migrate, verify и др.); инициация MigrationPipeline.	Migrator.Core, вызываются рендереры и парсеры.
<i>Migrator.Core</i>	Ядро: конвейер миграции (MigrationPipeline), модели действий и отчётов, интерфейсы IParser/IRenderer/IAdapter.	Независим от конкретного языка (использует лишь интерфейсы); используется всеми остальными.
<i>Migrator.SeleniumCSharp</i>	Парсинг C#-тестов Selenium: детектирование действий (SeleniumCSharpActionExtractor) и PageObject-структур; DefaultProjectAdapter для C#.	Зависит от Migrator.Core (IR), и от Migrator.Roslyn (разбор кода).
<i>Migrator.Roslyn</i>	Обход синтаксического дерева C#: RoslynTestFileParser, распознавание вызовов (Recognizer-ы).	Библиотека Microsoft.CodeAnalysis (Roslyn). Используется SeleniumCSharp.
<i>Migrator.PlaywrightDotNet</i>	Генерация C#-кода Playwright на основе IR.	Зависит от Migrator.Core (получает IR), использует шаблоны/синтаксис C#.
<i>Migrator.PlaywrightTypeScript</i>	Генерация TypeScript-кода Playwright.	Зависит от Migrator.Core (IR), использует шаблоны TS/JavaScript.

Модуль	Ответственность	Зависимости
<i>Adapters/Profiles</i>	Конфигурация маппинга (JSON/YAML профили страниц и методов) – применяется IProjectAdapter.	Используются в Core при адаптации IR.

Roadmap по этапам (описано выше в секции 9)

<abbr title="Диаграмма 1: Архитектура мигратора"><center>``mermaid graph LR subgraph CLI CLI[Migrator CLI] end subgraph Core MP[MigrationPipeline] IR[TestFileModel (IR)] IA[IProjectAdapter] / The core contains IR and pipeline / end subgraph Parser Roslyn[Roslyn Parser] SC[Selenium ActionExtractor] end subgraph Renderer DN[Playwright .NET Renderer] TS[Playwright TypeScript Renderer] end

```
CLI --> MP --> IR
MP --> IA
MP --> SC --> Roslyn
MP --> DN
MP --> TS
SC --> Roslyn
```

``</abbr>

<abbr title="Диаграмма 2: Примерный roadmap по этапам"><center>mermaid

timeline

title Roadmap по этапам

2026 : Анализ текущего решения; Сбор требований;

2026 : Проектирование IR/AST; Разработка архитектурных решений;

2027 : Имплементация нового парсера и генератора (пошагово по языкам);

2027 : Разработка тестов и настройка CI; Интеграционные проверки;

2028 : Финишная доводка; Документация и релиз. </abbr>

Источники: Документация к используемым технологиям (Roslyn [【23†L160-L168】](#) , JavaParser [【25†L83-L92】](#) , Python [ast](#) [【27†L64-L72】](#) , TypeScript API [【31†L1-L2】](#) , Go [go/parser](#) [【33†L195-L202】](#) , Rust [syn](#) [【35†L99-L107】](#)); а также статьи по дизайну компиляторов и трансляторов [【3†L23-L32】](#) [【7†L775-L784】](#) [【15†L52-L61】](#) .